Decision-Aware Memory Cards: Counterfactual-Inspired Context Selection and Compression for Tool-Using LLM Agents

Xinyu Guan[1], Qianyang Zhao[1], and Yuming Deng[1]

Alibaba Group, China
guanhan.gxy@alibaba-inc.com
zhaoqianyang.zqy@alibaba-inc.com
finaldreamer@qq.com
Corresponding author: Xinyu Guan.

Abstract. Modern large language model (LLM) agents do not simply need longer contexts; they need decision-relevant evidence at the moment of action. We study decision-aware context selection: ranking retrieved files, tests, traces, rules, and memories by their expected effect on an agent's next action rather than by semantic similarity alone. We present the Counterfactual-Inspired Context Layer (CICL), which builds an instance context graph, estimates decision-oriented utility for candidate units, and compresses selected evidence into typed memory cards. The same schema can be instantiated with hosted LLM judges, local surrogates, or lightweight rankers, making the selection protocol auditable across model choices. On 50 SWE-bench Verified file-retrieval instances, Qwen3.6-Plus reranking of BM25 top-50 candidates improves hit@1 from 0.58 to 0.78 and MRR@10 from 0.634 to 0.790, with all 2,500 judgments parseable. Controlled diagnostics show that CICL identifies action-critical evidence: removing the top-utility semantic unit reduces F1 from 0.245 to 0.000. In selected-then-compressed mode, memory cards save 44.93 tokens per query while preserving selected evidence. CICL provides a practical layer for measuring, ranking, and compressing decision-critical context for tool-using agents. Code is available at https://github.com/stephen-guan-researcher/CICL.

Keywords: LLM agents · Context selection · Counterfactual utility · Memory compression · LLM-as-a-judge diagnostics.

1 Introduction

A coding agent can fail even when the answer is already in the repository. The missing clue may be a failing test, a file-local convention, a stale trace to ignore, or a short rule buried among irrelevant snippets. More context does not fix this by itself. What matters is the evidence the agent uses before acting: the piece of context that changes its next command, edit, or test.

This issue becomes sharper as LLM agents move from text generation to tool use. Modern agents can interleave reasoning and action [1], call external tools

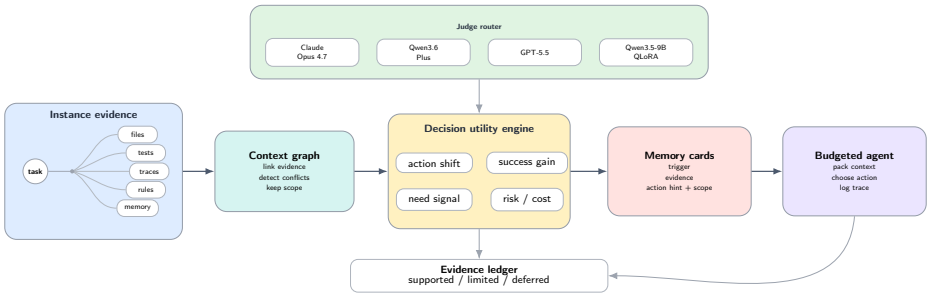

Fig. 1. CICL pipeline. The framework turns instance evidence into a graph, routes judge signals through a decision-utility engine, and packs memory cards for a budgeted agent. Model-dependent judge signals are reported separately rather than collapsed into one score.

[2], revise behaviour from feedback [3], and accumulate reusable skills [4]. In software engineering, systems such as SWE-agent [5] expose models to repositories, shell commands, tests, and edits. These abilities make context selection more important, not less: an agent acting on the wrong evidence can fail quickly and confidently. Recent coding-agent context benchmarks make this bottleneck explicit [6,7].

We therefore study context selection as a decision problem. Standard retrieval asks which text is most similar to the task. An agent instead needs evidence that can affect the next step: by changing an action, supporting a better outcome, becoming necessary for success, or avoiding a harmful choice. CICL instantiates this idea as a budgeted context layer that separates three roles: candidate generation, decision-oriented scoring, and compact context packaging. The same schema can be instantiated with hosted LLMs, local surrogates, or lightweight rankers, so the protocol remains auditable across model choices. Figure 1 gives an overview.

The paper asks whether this decision-aware signal helps rank agent context and whether the schema remains usable when the judge changes. We test this with SWE-bench Verified file retrieval, controlled removal diagnostics, compression experiments, and judge-agreement checks. The results show gains in real-code file retrieval and token-efficient context packing, while also exposing clear limits: strong baselines remain competitive, and the evidence is file-level rather than end-to-end patch success. This positioning keeps the claims safely focused on context selection rather than full agent repair or deployment.

Contributions. (1) We frame agent context selection as a decision-time intervention and formalise a four-component utility for action-critical evidence. (2) We introduce decision-aware memory cards and a graph-based assembly pipeline for packing action-oriented context. (3) We evaluate the same judgment schema across hosted LLMs, Qwen-family judges, lightweight rankers, and a local Qwen-QLoRA surrogate. (4) We provide controlled diagnostics identifying where the

framework works, where LLM-based reranking improves open benchmark retrieval, and where baselines remain stronger.

2 Related Work

LLM agent research has broadened evaluation from isolated responses to systems that coordinate actions, use tools, and preserve state over time. AutoGen and ChatDev show how model roles can be organised into collaborative software-development workflows [8,9]. For repository-level coding, Agentless and OpenHands cover complementary designs: strong non-agent repair pipelines and open infrastructure for executing coding agents [10,11]. Memory-oriented benchmarks show that agent state must be tested across sessions, turns, and self-evolving updates [12,13,14]. These works establish settings where available context matters, but they largely evaluate complete agent systems or benchmark outcomes. CICL treats this pre-action context choice as the unit of study rather than evaluating a complete agent architecture.

External context for agents is usually obtained through retrieval and then placed in the prompt. Dense retrieval and vector-search systems provide scalable candidate generation [15,16]. RAG conditions generation on retrieved evidence [17], while Atlas and HyDE improve retrieval through retrieval-scale pretraining and hypothetical documents [18,19]. Self-RAG makes this process more selective by reflecting on when retrieved passages are useful [20]. Long-context evaluations add a complementary caution: information included in a long input may still be missed or underused [21,22]. These methods improve access to evidence or test whether evidence can be used once supplied. CICL operates one step earlier in the pipeline, at the ranking stage that decides which retrieved units should reach the agent.

Context management for long-horizon agents creates a related selection pressure. AutoContext, ACE, and ACON adapt, evolve, or compress context for agentic settings [23,24,25]. Prompt compression methods reduce input length by preserving salient tokens, spans, or segments [26,27,28]. Contribution-aware methods ask which units help or harm downstream behaviour: CMI studies causal memory selection [29], and RepoShapley estimates repository-level contribution for code completion [30]. CICL uses these mechanisms for a narrower purpose: selecting compact context units whose value is measured by their role in the agent's decision process, not only by relevance or compression salience.

3 Method

3.1 Decision-Aware Context Selection

Consider an agent policy π acting on tasks $x \in \mathcal{X}$. At each decision step the agent receives a context block $C \subseteq \mathcal{U}$, where $\mathcal{U}$ is the pool of candidate units produced by the instance graph. Unlike BM25 [31], DPR [32], or ColBERT [33], CICL treats selected evidence as an intervention. A relevance selector solves

$C^{\text{rel}} = \arg\max_{\text{tok}(C)\le B} \sum_{c\in C} \text{sim}(c,x)$ under token budget B. CICL replaces sim with a decision-time utility $U(c,x)$ that estimates whether c would change the next action or expected success under the agent policy.

3.2 Counterfactual-Inspired Utility

Let C^- denote the context assembled before considering candidate c, and let $C^+ = C^- \cup \{c\}$. With $\pi(a \mid x, C)$ denoting the conceptual next-action distribution under context C, CICL decomposes utility into four components. For hosted LLM agents, we do not assume direct access to this distribution: π defines the target quantities, while CICL estimates the components using simulator probes, structured LLM judgments, or learned rankers under the same schema:

$$\begin{aligned} \Delta_{\text{act}}(c,x) &= \mathbb{E}\big[\,\mathbf{1}\{\arg\max_a \pi(a \mid x, C^+) \neq \arg\max_a \pi(a \mid x, C^-)\}\,\big] \\ \Delta_{\text{out}}(c,x) &= \mathbb{E}\big[\,V(x,C^+) - V(x,C^-)\,\big] \\ N(c,x) &= \Pr\big[\,\text{success}(x,C^+) = 1 \wedge \text{success}(x,C^-) = 0\,\big] \\ R(c,x) &= \Pr\big[\,c \text{ induces negative transfer on } x\,\big]. \end{aligned} \tag{1}$$

Here, V denotes an expected success score. $N(c,x)$ is a necessity-style proxy for cases where adding c changes failure into success, and $R(c,x)$ captures negative transfer. CICL aggregates the components with fixed signed weights:

$$U(c,x) = \alpha\,\Delta_{\text{act}}(c,x) + \beta\,\Delta_{\text{out}}(c,x) + \gamma\,N(c,x) - \lambda\,R(c,x). \tag{2}$$

For judge-style outputs, let hats denote evaluator-produced estimates of the corresponding conceptual components. Eq. 3 is a fixed operational instantiation of Eq. 2: it plugs the judge fields into the same signed utility structure and adds a bounded cost penalty:

$$\begin{aligned} s(c,x) = {} & 0.34\,\widehat{\Delta}_{\text{act}}(c,x) + 0.26\,\widehat{N}(c,x) + 0.28\,\widehat{\Delta}_{\text{out}}(c,x) \\ & - 0.22\,\widehat{R}(c,x) - 0.08\,\widehat{\text{cost}}(c,x). \end{aligned} \tag{3}$$

In the implementation, $\widehat{\text{cost}}(c,x) = \min(1, \text{tok}(c)/1000 + 0.2\widehat{R}(c,x))$ for LLM-judged units, matching the released scorer. We keep this aggregation fixed across Claude Opus, Qwen, and GPT-5.5 via Codex judge runs to prevent ablation drift; deterministic proxy ablations use the same component signs with model-free estimates. The coefficients in Eq. 3 are pre-set heuristic weights, not learned from test labels. Component-removal ablations provide the current sensitivity evidence; equal-weight, random-weight, and learned-weight sweeps remain future work. Table 1 lists the judge output fields. We stress that "causal" here denotes counterfactual-inspired utility estimation rather than formal causal identification: the expectations above are not identified by any randomised intervention, and we defer a detailed discussion of this boundary to Section 6.

3.3 Instance Context Graph

For each repository or environment instance, CICL constructs a graph whose nodes correspond to files, symbols, task memories, rules, failures, and strategy records. Edges capture containment, similarity, conflict, precondition, and

Table 1. Eight-field judge schema used by Claude Opus and Qwen. The four utility fields estimate the hatted terms in Eq. 3; confidence is retained for audit and diagnostics, and token cost comes from context metadata.

Field	Role	Field	Role
`no_context_action`	action without unit	`with_context_action`	action with unit
`action_shift`	action change $[0, 1]$	`necessity`	dependence $[0, 1]$
`expected_outcome_uplift`	value gain $[0, 1]$	`negative_transfer_risk`	conflict risk $[0, 1]$
`reason`	short rationale	`confidence`	self-confidence $[0, 1]$

task-memory relations. The graph combines lexical and structural retrieval with one-hop neighbour expansion, which supports recovery of decision-relevant context even when lexical overlap with the query is weak. Each node is a context unit annotated with an identifier, instance id, type, source, content, token cost, and confidence score. Importantly, the graph never requires gold context identifiers for selection; gold ids appear only during offline evaluation and in oracle baselines. We audit all method-facing artifacts for gold-label leakage and include the audit script in the reproducibility package.

3.4 Decision-Aware Memory Cards

CICL compiles selected units into compact memory cards with five mandatory fields—*trigger* (when to consult), *evidence* (supporting clue), *action hint* (next-action verb), *failure-if-ignored* (risk if skipped), and *scope* (applicable boundary)—plus a diagnostic *decision-utility estimate* for ordering. The format prioritises decision usefulness over exhaustive semantic fidelity. Generic prompt compression often optimises token- or sentence-level importance, as in LLMLingua [26] and LongLLMLingua [27], or filters by salience [28]; CICL stores typed decision fields. A deterministic structural audit checks required-field completeness, action-verb presence, compression ratio, and absence of placeholder text.

3.5 Budget-Aware Assembly

At inference time, CICL retrieves candidate units, expands graph neighbours, scores candidates via U or its operational estimate s, and packs the highest-utility evidence under a fixed token budget. We distinguish post-selection compression, which first selects ids and then compresses their text, from pre-budget compression, which changes candidate costs before packing and can therefore change the selected ids. We report the two modes separately to avoid conflating compression gains with changes in selection.

3.6 Opus-Assisted Annotation and an Open-Weights Surrogate Judge

Claude Opus 4.7 is used to assist a human-designed context-utility annotation workflow. These provider-assisted diagnostic annotations are not human gold labels; they train small CICL context rankers rather than a reproduced teacher:

a 25-feature pairwise linear ranker and a two-layer MLP. All features are available at inference time, including proxy utility components, retrieval and lexical-overlap scores, unit type, token cost, confidence, history success, and graph signals such as conflict degree and source/task overlap. Rankers are trained from within-task positive–negative candidate pairs, following preference-learning style supervision used in instruction tuning [34] and summarisation from preferences [35], and are then used only for context ordering. After excluding features unavailable at inference time, the cleaned linear ranker reaches 0.939 pairwise accuracy on the v1 suite. Provider-side `llm_*` features are not used by any reported selector.

As a local open-weights alternative we fine-tune Qwen3.5-9B with QLoRA on two 16 GB V100 GPUs for the same eight-field schema, following LoRA adaptation [36] and QLoRA quantized fine-tuning [37]. We release the adapter at https://huggingface.co/XinyuGuan/CICL. It must be loaded with `Qwen/Qwen3.5-9B` and is used here as an agreement surrogate rather than a standalone judge or a Claude Opus replacement. Training uses the released v1 Opus-assisted SFT split (1,400 examples; 1,256 train and 144 validation examples, split by task); evaluation measures selection-level agreement via top-k Jaccard and Spearman ρ on 710 candidates from 25 base tasks. This keeps deterministic, Opus-assisted, lightweight-ranker, and Qwen-surrogate scores auditable as separate components.

4 Experimental Setup

4.1 Tasks and Benchmarks

Table 2 gives the minimal data map. SWE-bench Verified is the main real-code retrieval benchmark [38]; synthetic suites isolate mechanism; RepoBench-R tests compression [39]. CodeSearchNet motivates the broader semantic code-search setting that these repository-level retrieval tasks inherit [40].

Table 2. Dataset summary with scale, use, and evaluation scope.

Source	Use	Scale	Evaluation scope
SWE-bench Verified	Public benchmark-derived file-retrieval setting	50 instances; 2,500 Qwen judgments	File retrieval only; no patch success
Synthetic v1/v3	Controlled mechanism tests	250 eval tasks each; 1,400/1,800 labels	Ranking sanity, removal, and budget effects
RepoBench-R	Real-code compression	100 tasks; budgets 60–400	Compression, not repair success
Judge/ranker pilots	Judge substitution diagnostics	200 Opus-assisted labels; 710 Qwen-agreement candidates; 250 GPT-5.5 judgments	Surrogate and provider checks only

4.2 Methods Compared

We compare CICL and CICL_Distilled with NoContext, FullContext, VanillaRAG, GraphMemory, SummaryMemory, SelfGeneratedExamples, AutoCon-

textKG, and OracleGoldContext. The last uses gold ids only as an upper bound and is never employed as a selector elsewhere. We additionally report seven CICL ablations removing individual scoring components. In the synthetic result tables, CICL_Distilled denotes the learned student ranker family; rows marked CICL_Dist. (linear) and CICL_Dist. (MLP) distinguish the linear and non-linear ranker variants.

4.3 Metrics

For each task we report simulated success rate, context precision/recall/F1 against gold ids, mean reciprocal rank (MRR) of the first gold id in the selection, average tokens consumed, and average tool calls. For SWE-bench file retrieval, hit@1 measures whether the top-ranked file is gold, coverage measures whether any gold file appears in the candidate pool, R@k is file-level recall over gold files in the top-k, MRR@10 uses the first gold file within the top 10, and gold rank reports the mean first-gold rank over covered instances. In settings where paired task-level deltas are available, compression-suite experiments additionally employ paired bootstrap tests with 2,000 resamples for delta metrics and report 95% confidence intervals; bootstrap pairs are matched per task id. The executable pilots further report patch success and harmful-selection rates.

4.4 Implementation Details

The deterministic simulator, the heuristic CICL ranker, the AutoContextKG selector, and the compression pipeline are implemented in pure Python with fixed random seeds. The Opus-assisted annotation pipeline queries Claude Opus 4.7 with provider-supported decoding settings using a counterfactual prompt template that enforces the eight-field JSON schema. The synthetic linear rankers are trained for 80 epochs with the pairwise logistic trainer (learning rate 0.08, L2 5×10^{-4} in the public script); the real-code pilot uses the same trainer for 100 epochs. The MLP rankers use the same cleaned examples but optimise a BPR pairwise loss for 120 epochs with AdamW (learning rate 10^{-3}, batch size 256, weight decay 10^{-4}, dropout 0.1). The Qwen3.5-9B QLoRA judge is trained on two 16 GB V100 GPUs and uses rank $r = 8$, $\alpha = 16$, dropout 0.05, target modules `q_proj, k_proj, v_proj, o_proj`, effective batch size 16 (batch 1×16 gradient accumulation steps), one epoch, lr $= 2 \times 10^{-4}$, bf16 disabled (V100 compatibility), and 4-bit NF4 base-weight quantisation. Local generation, training, and evaluation scripts use fixed seeds. Decoding used deterministic or provider-default settings where supported, and all external annotation runs keep archived JSON outputs; because hosted LLM backends can drift, reported numbers are tied to the released artifacts rather than to re-querying the provider. The GPT-5.5 via Codex path uses the same code-retrieval prompt and eight-field schema on the first five SWE-bench Verified instances; it is reported as a small provider check, not as a full retrieval benchmark.

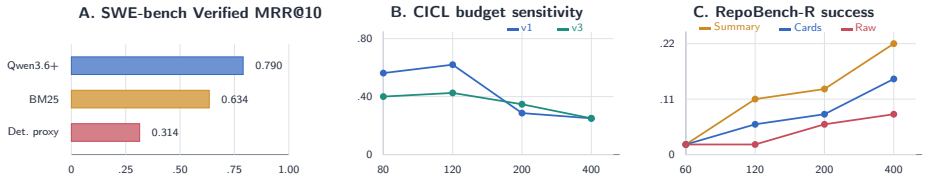

Fig. 2. Compact result overview. Panel A foregrounds the SWE-bench Verified retrieval result; Panels B and C show the main controlled budget trend and RepoBench-R compression boundary.

5 Results

5.1 SWE-bench Verified File Retrieval

We begin with the open benchmark most likely to matter for coding agents: SWE-bench Verified file-level retrieval. This setting does not measure patch success, but it asks whether the selector can place the target file high enough for a downstream agent to act. On 50 instances (Table 3), BM25/HybridRAG reaches hit@1 0.58 and MRR@10 0.634. The older deterministic decision-utility proxy is weaker than BM25, but direct Qwen3.6-Plus judgments over the top-50 file pool raise hit@1 to 0.78 and MRR@10 to 0.790, with 2500/2500 parseable judgments. The result is the paper's main positive real-code finding: decision-aware judging provides a useful reranking signal when the judge can read the candidate evidence well. Figure 2 places this open benchmark next to the controlled mechanism and compression trends.

Table 3. SWE-bench Verified file-level retrieval (50 instances), sorted by non-oracle MRR@10. Direct Qwen3.6-Plus judgments re-rank top-50 candidates; the metric is retrieval quality and not a patch result.

Method	Hit@1	Coverage	R@5	R@10	MRR@10	Gold rank
Qwen CausalRerank	**0.78**	0.80	**0.654**	**0.654**	**0.790**	**1.03**
Qwen CausalHybrid	0.70	**0.82**	0.624	0.634	0.733	1.88
BM25	0.58	0.80	0.554	0.634	0.634	2.43
HybridRAG	0.58	0.80	0.554	0.634	0.634	2.43
Deterministic CausalRerank	0.26	0.56	0.274	0.324	0.314	5.25
HashedEmbedding	0.18	0.30	0.160	0.160	0.192	5.07
Random	0.00	0.04	0.000	0.007	0.002	10.50
OracleGoldContext	1.00	1.00	0.824	0.824	1.000	1.00

We also ran the same prompt and schema through GPT-5.5 via Codex on the first five SWE-bench Verified instances (250 candidate judgments). All judgments parsed; on this five-instance slice, CausalRerank, CausalHybridRerank, BM25, and HybridRAG all obtain hit@1 and MRR@10 of 0.60. We use this only to check that the schema can run through another provider; the 50-instance Qwen3.6-Plus run is the main retrieval comparison.

5.2 Ablation

Tables 4 and 5 report the two ablation diagnostics most directly tied to the selection mechanism: removing selected evidence and varying the token budget.

The clearest controlled signal is the top-utility removal diagnostic. Removing the highest-scoring unit collapses v3 context F1 from 0.245 to 0.000 and MRR from 0.980 to 0.000; random removal is much less destructive (F1 0.205). The supplement adds the same removal check on v1 and a compression-order split.

Table 4. Top-utility removal ablation on synthetic v3 (250 tasks, budget 400). Removing the highest-utility unit collapses F1 and MRR to zero.

Condition	F1	MRR	Precision	Recall
CICL_full	0.245	0.980	0.196	0.327
CICL_remove_random	0.205	0.716	0.179	0.239
CICL_remove_top	0.000	0.000	0.000	0.000

The second controlled pattern is an inverted-U over token budgets (Table 5). CICL peaks at budget 120 on both suites (v1 0.620, v3 0.425) and then decays as larger budgets admit more distractors. On v3, VanillaRAG remains stronger, so the budget result is a limited positive result rather than evidence of dominance.

Table 5. Context F1 across token budgets (250 tasks each). Rows are sorted by non-oracle peak strength. CICL peaks at budget 120 on both suites; the inverted-U is driven by precision loss at larger budgets. Bold marks each row's peak.

	Synthetic v1				Synthetic v3			
Method	80	120	200	400	80	120	200	400
AutoContextKG	0.706	**0.850**	0.671	0.481	**0.481**	0.406	0.441	0.305
VanillaRAG	0.523	0.562	**0.643**	0.459	0.465	**0.525**	0.477	0.373
CICL	0.562	**0.620**	0.286	0.250	0.400	**0.425**	0.347	0.250
CICL_Distilled	**0.414**	0.295	0.405	0.350	**0.305**	0.241	0.230	0.175

5.3 Main Synthetic Ranking Results

Table 6 reports the standard budget-120 comparison, sorted by average non-oracle F1 across the two suites. On v1, CICL is below AutoContextKG but above VanillaRAG and GraphMemory. On v3, semantic distractors change the ordering: VanillaRAG has the best F1 and AutoContextKG the best MRR, while CICL remains competitive but no longer dominates.

5.4 RepoBench-R Compression Suite

Table 7 reports the 100-task RepoBench-R Python compression suite. At budget 120, memory-card compression improves over raw selection in success ($0.02 \rightarrow 0.06$) and context recall ($+0.04$, 95% CI $[0.01, 0.08]$, $p = 0.016$ by matched paired bootstrap with 2,000 resamples). Generic extractive summarisation is nevertheless stronger at the same budget (0.11 success; summary-vs-card recall delta $+0.05$, 95% CI $[0.01, 0.10]$, $p = 0.009$). In selected-then-compressed mode, memory cards save 44.93 tokens per query over raw selection (95% CI $[41.05, 48.74]$, $p < 10^{-3}$) with identical selected ids. The supported result is therefore specific: cards can save tokens and improve raw budget fitting, but generic summaries are the stronger baseline on this starter slice.

Table 6. Main 250-task evaluation at budget 120, sorted by average non-oracle F1 across v1/v3. Bold = best non-oracle per metric.

	Synthetic v1 (structural)			Synthetic v3 (semantic)		
Method	F1	MRR	Tokens	F1	MRR	Tokens
AutoContextKG	**0.850**	**0.984**	94.3	0.406	**1.000**	98.4
VanillaRAG	0.562	0.603	107.3	**0.525**	0.593	103.6
CICL	0.620	0.817	117.8	0.425	0.896	117.2
CICL_Dist. (MLP)	0.328	0.277	111.2	0.334	0.273	113.0
GraphMemory	0.511	0.654	116.8	0.303	0.617	117.0
CICL_Dist. (linear)	0.295	0.302	110.6	0.241	0.226	113.7
SelfGenExamples	0.138	0.338	98.6	0.124	0.366	101.9
FullContext	0.020	0.037	115.0	0.027	0.045	114.0
SummaryMemory	0.010	0.018	99.8	0.007	0.014	102.4
NoContext	0.000	0.000	0.0	0.000	0.000	0.0
OracleGold	1.000	1.000	89.0	1.000	1.000	45.0

Table 7. RepoBench-R 100-task compression sweep (means across 100 tasks); rows are sorted by success and then F1 within each budget.

Budget	Method	Success	F1	Tokens	Selected	Ratio
60	Memory card	0.02	0.014	49.8	2.95	0.51
60	Raw	0.02	0.013	47.5	2.02	1.00
60	Summary	0.02	0.009	55.2	2.63	0.26
120	Summary	**0.11**	**0.030**	115.3	5.31	0.27
120	Memory card	0.06	0.023	103.6	4.43	0.38
120	Raw	0.02	0.007	106.1	2.88	1.00
200	Summary	0.13	0.022	193.2	8.90	0.24
200	Memory card	0.08	0.022	186.7	5.92	0.42
200	Raw	0.06	0.020	186.0	4.09	1.00
400	Summary	**0.22**	**0.030**	335.1	14.51	0.19
400	Memory card	0.15	0.027	380.8	10.08	0.41
400	Raw	0.08	0.020	374.3	5.52	1.00

5.5 Opus-Assisted Rankers and Qwen Selection Agreement

We collected 1 400 Claude Opus 4.7-assisted counterfactual judgments on the v1 base tasks and trained lightweight rankers. The linear ranker reaches 0.939 pairwise accuracy but remains below the heuristic on downstream F1; the MLP improves over linear on both suites (v3 F1 0.241 $\rightarrow$ 0.334). A separate 200-label RepoBench-R pilot gives only a small heldout signal, so we treat real-code ranker training as preliminary. The Qwen3.5-9B QLoRA judge is used only as an agreement diagnostic on 710 synthetic-distribution candidates (Table 8), not as a standalone judge.

Table 8. Qwen3.5-9B QLoRA judge vs. Claude Opus 4.7 (710 candidates, 25 tasks).

Metric	Value
JSON parse rate	1.000
Avg. top-5 Jaccard vs. Opus	0.592
Avg. Spearman ρ vs. Opus	0.379
MAE (max across 5 numeric fields)	0.059

5.6 Additional Diagnostics

A two-task toy patch-and-test pilot checks that selected context can pass through an executable patch loop: CICL, VanillaRAG, and AutoContextKG reach patch success 1.00, while NoContext reaches 0.00. Three Astropy patch-generation smoke checks test formatting but are not official SWE-bench passes. In harmful-context stress tests, CICL often selects stale units (70.8% on v1; 68.8% on v3), but ranks the gold unit ahead of them (harmful-before-gold = 0.00). A structural audit of memory cards reports field completeness 1.000, action hints 1.000, compression success 0.948, and token ratio 0.486.

6 Limitations

CICL has several scoped limitations. First, the score is counterfactual-inspired rather than a formally identified causal effect; its components are estimated by simulators, LLM judges, or rankers, and the reported scorer uses fixed heuristic weights. Second, the main real-code result is SWE-bench Verified file-level retrieval, not official patch success; the patch experiments are small smoke checks. Third, judge quality matters: hosted models may drift, and the Qwen-QLoRA result measures agreement on the synthetic Opus-assisted distribution. Finally, typed cards can lose lexical detail, and strong baselines remain competitive.

7 Conclusion

CICL treats context selection for tool-using LLM agents as a decision-aware problem: useful context is evidence that should affect the next step, not simply text that is similar to the task. The framework separates candidate generation, decision-oriented scoring, and compact memory-card packaging under an auditable judge schema.

Across the experiments, this framing yields a useful but bounded signal. LLM-based reranking improves SWE-bench Verified file retrieval, removal diagnostics show that top-scored units can be action-critical, and selected-then-compressed cards reduce token use while preserving selected evidence. The negative results are equally important: generic summaries can be stronger for code completion, compact rankers do not yet replace the heuristic selector, and the evidence remains file-level rather than end-to-end repair success. Future work should scale real-code annotations, learn adaptive utility weights, combine structured cards with lexical snippets, and evaluate the selector inside full agent rollouts.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: The Eleventh International Conference on Learning Representations (2023), https://openreview.net/forum?id=WE_vluYUL-X
2. Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. In: Advances in Neural Information Processing Systems. vol. 36, pp. 68539–68551 (2023), https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html
3. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: Advances in Neural Information Processing Systems. vol. 36, pp. 8634–8652 (2023), https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
4. Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., Anandkumar, A.: Voyager: An open-ended embodied agent with large language models. Transactions on Machine Learning Research (2024), https://openreview.net/forum?id=ehfRiF0R3a
5. Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: SWE-agent: Agent-computer interfaces enable automated software engineering. In: Advances in Neural Information Processing Systems. vol. 37, pp. 50528–50652 (2024), https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
6. Li, H., et al.: Contextbench: A benchmark for context retrieval in coding agents (2026), https://arxiv.org/abs/2602.05892
7. Zhu, J., et al.: Swe context bench: A benchmark for context learning in coding (2026), https://arxiv.org/abs/2602.08316
8. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., Wang, C.: AutoGen: Enabling next-gen llm applications via multi-agent conversations. In: First Conference on Language Modeling (2024), https://openreview.net/forum?id=BAakY1hNKS
9. Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., Sun, M.: ChatDev: Communicative agents for software development. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 15174–15186. Association for Computational Linguistics, Bangkok, Thailand (2024). https://doi.org/10.18653/v1/2024.acl-long.810, https://aclanthology.org/2024.acl-long.810/
10. Xia, C.S., Deng, Y., Dunn, S., Zhang, L.: Demystifying llm-based software engineering agents. Proceedings of the ACM on Software Engineering **2**(FSE), 801–824 (2025). https://doi.org/10.1145/3715754, https://doi.org/10.1145/3715754
11. Wang, X., Li, B., Song, Y., Xu, F.F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H.H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., Neubig, G.: OpenHands: An open platform for ai software developers as generalist agents. In: The Thirteenth International Conference on Learning Representations (2025), https://openreview.net/forum?id=OJd3ayDDoF
12. He, Z., Wang, Y., Zhi, C., Hu, Y., Chen, T.P., Yin, L., Chen, Z., Wu, T.A., Ouyang, S., Wang, Z., Pei, J., McAuley, J., Choi, Y., Pentland, A.: MemoryArena:

Benchmarking agent memory in interdependent multi-session agentic tasks. In: Proceedings of the 43rd International Conference on Machine Learning (2026), https://arxiv.org/abs/2602.16313, accepted as a regular paper
13. Hu, Y., Wang, Y., McAuley, J.: Evaluating memory in LLM agents via incremental multi-turn interactions. In: The Fourteenth International Conference on Learning Representations (2026), https://openreview.net/forum?id=DT7JyQC3MR
14. Wang, Y., et al.: Evomembench: Benchmarking agent memory from a self-evolving perspective (2026), https://arxiv.org/abs/2605.18421
15. Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., Grave, E.: Unsupervised dense information retrieval with contrastive learning. Transactions on Machine Learning Research (2022), https://openreview.net/forum?id=jKN1pXi7b0
16. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with gpus. IEEE Transactions on Big Data **7**(3), 535–547 (2019). https://doi.org/10.1109/TBDATA.2019.2921572, https://doi.org/10.1109/TBDATA.2019.2921572
17. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive nlp tasks. In: Advances in Neural Information Processing Systems. vol. 33 (2020), https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html
18. Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., Grave, E.: Atlas: Few-shot learning with retrieval augmented language models. Journal of Machine Learning Research **24**(251), 1–43 (2023), https://jmlr.org/papers/v24/23-0037.html
19. Gao, L., Ma, X., Lin, J., Callan, J.: Precise zero-shot dense retrieval without relevance labels. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 1762–1777. Association for Computational Linguistics, Toronto, Canada (2023). https://doi.org/10.18653/v1/2023.acl-long.99, https://aclanthology.org/2023.acl-long.99/
20. Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: The Twelfth International Conference on Learning Representations (2024), https://openreview.net/forum?id=hSyW5go0v8
21. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics **12**, 157–173 (2024). https://doi.org/10.1162/tacl_a_00638, https://aclanthology.org/2024.tacl-1.9/
22. Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., Li, J.: LongBench: A bilingual, multitask benchmark for long context understanding. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 3119–3137. Association for Computational Linguistics, Bangkok, Thailand (2024). https://doi.org/10.18653/v1/2024.acl-long.172, https://aclanthology.org/2024.acl-long.172/
23. Cai, K., et al.: Autocontext: Instance-level context learning for llm agents (2025), https://arxiv.org/abs/2510.02369
24. Zhang, Q., Hu, C., Upasani, S., Ma, B., Hong, F., Kamanuru, V., Rainton, J., Wu, C., Ji, M., Li, H., Thakker, U., Zou, J., Olukotun, K.: Agentic context engineering: Evolving contexts for self-improving language models. In: The Fourteenth International Conference on Learning Representations (2026), https://openreview.net/forum?id=eC4ygDs02R

25. Kang, M., Chen, W.N., Han, D., Inan, H.A., Wutschitz, L., Chen, Y., Sim, R., Rajmohan, S.: ACON: Optimizing context compression for long-horizon LLM agents. In: Lifelong Agent Workshop at the Fourteenth International Conference on Learning Representations (2026), https://openreview.net/forum?id=x0alNh5o8v
26. Jiang, H., Wu, Q., Lin, C.Y., Yang, Y., Qiu, L.: LLMLingua: Compressing prompts for accelerated inference of large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. pp. 13358–13376. Association for Computational Linguistics, Singapore (2023). https://doi.org/10.18653/v1/2023.emnlp-main.825, https://aclanthology.org/2023.emnlp-main.825/
27. Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.Y., Yang, Y., Qiu, L.: LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 1658–1677. Association for Computational Linguistics, Bangkok, Thailand (2024). https://doi.org/10.18653/v1/2024.acl-long.91, https://aclanthology.org/2024.acl-long.91/
28. Li, Y., Dong, B., Guerin, F., Lin, C.: Compressing context to enhance inference efficiency of large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. pp. 6342–6353. Association for Computational Linguistics, Singapore (2023). https://doi.org/10.18653/v1/2023.emnlp-main.391, https://aclanthology.org/2023.emnlp-main.391/
29. Srivastava, S.S.: Causal intervention-based memory selection for long-horizon llm agents (2026), https://arxiv.org/abs/2605.17641
30. Huo, Y., Zeng, K., Zhang, S., Lu, Y., Yang, C., Guo, Y., Tang, X.: RepoShapley: Shapley-enhanced context filtering for repository-level code completion. In: Findings of the Association for Computational Linguistics: ACL 2026 (2026), https://arxiv.org/abs/2601.03378, in press
31. Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval **3**(4), 333–389 (2009). https://doi.org/10.1561/1500000019, https://doi.org/10.1561/1500000019
32. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 6769–6781. Association for Computational Linguistics, Online (2020). https://doi.org/10.18653/v1/2020.emnlp-main.550, https://aclanthology.org/2020.emnlp-main.550/
33. Khattab, O., Zaharia, M.: ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 39–48. Association for Computing Machinery (2020). https://doi.org/10.1145/3397271.3401075, https://doi.org/10.1145/3397271.3401075
34. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P.F., Leike, J., Lowe, R.: Training language models to follow instructions with human feedback. In: Advances in Neural Information Processing Systems. vol. 35, pp. 27730–27744 (2022), https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html
35. Stiennon, N., Ouyang, L., Wu, J., Ziegler, D.M., Lowe, R., Voss, C., Radford, A., Amodei, D., Christiano, P.F.: Learning to summarize with human feedback. In: Advances in Neural Information Processing Systems.

vol. 33, pp. 3008–3021 (2020), https://proceedings.neurips.cc/paper/2020/hash/1f89885d556929e98d3ef9b86448f951-Abstract.html
36. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: The Tenth International Conference on Learning Representations (2022), https://openreview.net/forum?id=nZeVKeeFYf9
37. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: Efficient finetuning of quantized LLMs. In: Advances in Neural Information Processing Systems. vol. 36, pp. 10088–10115 (2023), https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html
38. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: Can language models resolve real-world github issues? In: The Twelfth International Conference on Learning Representations (2024), https://openreview.net/forum?id=VTF8yNQM66
39. Liu, T., Xu, C., McAuley, J.: Repobench: Benchmarking repository-level code autocompletion systems. In: The Twelfth International Conference on Learning Representations (2024), https://openreview.net/forum?id=pPjZIOuQuF
40. Husain, H., et al.: CodeSearchNet challenge: Evaluating the state of semantic code search. In: NeurIPS Workshop on Machine Learning for Software Engineering (2019), https://arxiv.org/abs/1909.09436